



푸딩툰(React + Next.js) 안정화 & 뮤직(OST) 확장

1. 프로젝트 개요

- **프로젝트명:** 푸딩툰 OST(뮤직) 서비스 신규 구축(1차) 및 2차 고도화
- **기간:** 2026.01 ~ 2026.02
- **플랫폼/환경:** Next.js Web + Flutter WebView(App)
- **담당 범위:** OST 플레이어/페이지군 구축, 운영 도구(관리자) 연동, QA/심사 대응, GA4/GTM 분석 체계 구축

2. 요구사항 및 나의 역할

요구사항 요약

- 기존 서비스에 **OST 감상(뮤직) 경험**을 추가하고, 웹/앱(WebView)에서 동일한 사용 흐름을 제공
- 서비스 오픈을 위한 **QA 이슈 대응** 및 **iOS 심사 이슈(연령 확인/거절) 대응**
- 런칭 이후 제품 개선을 위해 **퍼널 분석 및 A/B 비교**가 가능한 이벤트/데이터 수집 체계 정비

담당 역할

- **OST 기능/플로우 설계 및 구현 리딩:** 진입 동선, 모달/페이지 정책, 보이스툰 플레이어와의 충돌 방지 전략 정리
- **핵심 기능 개발:** 재생, 미니플레이어, 상세/리스트, 플레이리스트 편집, 스트리밍 등 주요 기능을 단계적으로 확장
- **운영/QA/심사 대응:** 이슈 트래킹 및 재현-수정-검증 루프 운영, 스토어 심사 대응(노출/제외 포함)
- **데이터 분석 체계 구축:** 하이브리드 환경에서 앱/웹 데이터 분리 및 전환 퍼널 이벤트 설계

3. 기술 스택 및 아키텍처

항목	사용 기술 / 라이브러리
프레임워크/런타임	Next.js Web, Flutter WebView(App)
플레이어 상태 관리	전역 Store(useAudioPlayerStore) 기반 재생 상태/UI 동기화
플레이리스트 편집	Web: dnd-kit / App: Flutter(ReorderableListView + Dismissible)
브릿지/동기화	Flutter ↔ Web 양방향 브릿지 이벤트(재생/삭제/순서변경) + currentIndex 동기화
분석	GA4 + GTM (is_app / platform_type 기반 분리, page_view 복구)

4. 주요 기능

기능	설명
OST 재생/플레이어	재생/일시정지, 탐색(드래그), 셔플/반복, 볼륨, 키보드 컨트롤, 재시도(최대 3회), 온라인/오프라인 감지
미니플레이어(전역)	재생 시작 시 노출, 플레이어 페이지 진입 시 숨김, 닫기 시 reset 등 상태 전이 정책 정의
플레이리스트 편집	Web(PC) DnD 편집 + App(iOS/Android) 네이티브 편집 UI로 분기
플랫폼 동기화	Flutter ↔ Web 브릿지로 재생/삭제/순서 변경 동기화(재생 인덱스 포함)
분석/퍼널	로그인 전환, 코인 충전 퍼널, 배너/검색/팔로우 등 주요 이벤트 표준화 및 확장
심사/QA 대응	iOS 심사 이슈(연령 확인/거절) 대응, QA 이슈 우선순위화 및 운영 안정화

5. 주요 성과

성과	설명
웹/iOS/안드로이드 공통 뮤직 플레이어 엔진 구축	URL 파라미터 기반 플레이어를 제거하고 전역 Store(useAudioPlayerStore) 기반으로 재생 경험을 통일. 미니플레이어 상태 전이 정의 및 핵심 재생 기능/안정성(재시도, 온라인/오프라인 감지, 키보드 컨트롤 등)까지 포함해 완성도 확보
웹/앱(Flutter WebView) 플레이리스트 편집 경험 구현 및 동기화	웹은 dnd-kit DnD 편집, 앱은 Flutter 네이티브 UI(ReorderableListView + Dismissible)로 분기. Flutter ↔ Web 브릿지로 재생/삭제/순서 변경을 연결하고 currentIndex까지 일관되게 동기화

성과	설명
앱 오픈 안정화(심사/QA 리스크 해소)	iOS 심사(연령 확인/거절) 이슈 소명·재심사 제출까지 대응. QA 이슈 우선순위로 운영 품질 안정화
퍼널·A/B 비교 가능한 GA4/GTM 체계 구축	is_app / platform_type 기반 GTM 동적 라우팅으로 웹/앱 데이터 분리 + page_view 복구. 로그인 전환, 코인 충전 퍼널 등 핵심 이벤트 표준화 및 주요 인터랙션 이벤트 확장
플레이어 UX 고도화	미니플레이어 상태 유지/동기화, 재생 중 곡 삭제 방지, 중복 담기 허용 등 정책 기반 UX로 이용 흐름 강화

6. 문제 해결 및 기술적 도전

✓ 하이브리드(Webview) 환경에서의 데이터 품질 문제

- 앱 유저 이벤트가 웹 GA4로 섞이거나, 앱 환경에서 page_view 차단으로 세션 공백이 발생하는 문제를 해결
- 앱 스토어 업데이트 없이 프론트 배포 + GTM 설정만으로 적용 가능하도록 설계해 리드타임을 단축

✓ 심사/운영 이슈가 동시에 발생하는 상황에서의 릴리즈 안정화

- 심사 이슈 대응과 QA 수정/재검증 루프를 병행 운영해 오픈 리스크를 최소화

7. 협업 및 커뮤니케이션

- 애자일 기반 싱크 운영: 기획/디자인/백엔드와 주기적으로 싱크를 돌리며 요구사항 변경을 빠르게 반영하고, 플레이어 정책(재생/듣기/담기, 미니플레이어 동기화 등)과 데이터 수집 정책(GTM 이벤트 정의)을 문서화해 정렬
- 릴리즈/QA 흐름 분리로 안정성 확보: Git 기반 협업에서 `dev → staging → main` 흐름으로 통합 테스트, QA, 운영 배포 단위를 분리해 리스크를 낮추고, Webview(iOS/Android) 환경까지 포함한 검증 루프를 운영
- 가이드/리뷰를 통한 품질 관리: 플레이어 엔진과 GTM 설계처럼 영향 범위가 큰 변경은 구현 의도와 사용 규칙(이벤트 네이밍/파라미터, 플레이리스트 정책)을 정리하고 리뷰를 통해 팀 내 일관성을 유지

8. 개발 결과 및 회고

결과

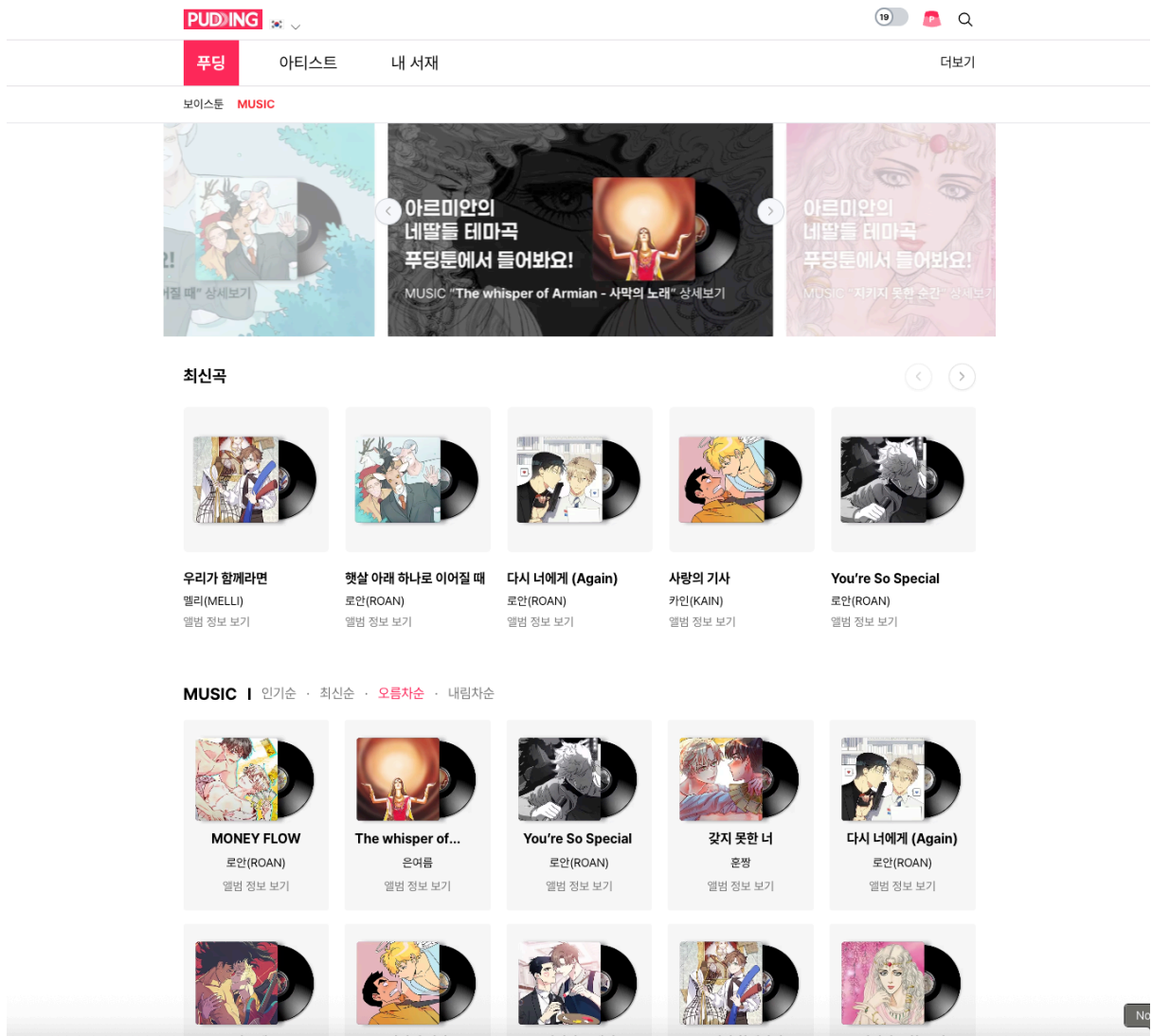
- Web/앱(WebView) 공통으로 동작하는 **뮤직 플레이어 엔진과 플레이리스트 편집 경험**을 구축하고, 플랫폼 제약(WebView 터치/드래그)을 분석해 앱은 Flutter 네이티브 UI로 분기하여 iOS/Android 완성도를 확보
- **GTM 동적 라우팅 + page_view 복구**를 포함한 하이브리드 트래킹 구조를 정비해, 앱/웹 데이터 품질을 개선하고 **퍼널 분석 및 A/B 비교**가 가능한 분석 기반을 제품 레벨로 정착
- 심사/QA/운영 이슈를 우선순위로 관리하며 재현-수정-검증 루프를 운영해 출시 리스크를 최소화

🙄 회고

- 플레이어 엔진과 GTM 설계는 서비스 전역에 영향을 주는 변경이라, “일단 동작”보다 **정책(정의) → 구조(공통화) → 확장(이벤트/기능 추가)** 순서로 설계하는 접근이 효과적이었음
- 하이브리드(WebView) 특성상 “웹에서 되는 구현”이 앱에서 그대로 보장되지 않아, 초기에 제약을 빠르게 검증하고(예: dnd-kit 터치), **플랫폼별 최적 해법(Flutter 네이티브 분기 + 브릿지 동기화)**으로 전환하는 것이 품질/일정 모두에 유리했음
- 향후에는 플레이어/분석처럼 공통 인프라 성격의 모듈을 더 표준화하고, 테스트 시나리오와 운영 가이드를 선제적으로 강화해 팀 전체 비용을 낮추는 방향으로 개선하고자 함

9. 스크린샷

- 앱 주요 화면별 캡처 이미지 (홈 / 기능 A / 기능 B 등)



이미지 1. 뮤직 메인 페이지

PUDING

19

푸딩

아티스트

내 서재

더보기

보이스톤

MUSIC

앨범 정보

[싱글]

별별별별 - 테마곡 (Ver.1)

장르 POP

참가 아티스트 로안(ROAN) >

발매사 Dobedub

별별별별

보이스톤 보러가기

수록곡(1)

▶ 듣기

≡ 담기

번호

곡정보

듣기

담기

1

로안(ROAN) / POP / 03:14

2026. 2. 4.

▶

≡

추천 앨범

최강의 냄새 - 테마곡 (Ver.1)

장르 POP

♡ 3

다크헤븐 - 테마곡 (Ver.1)

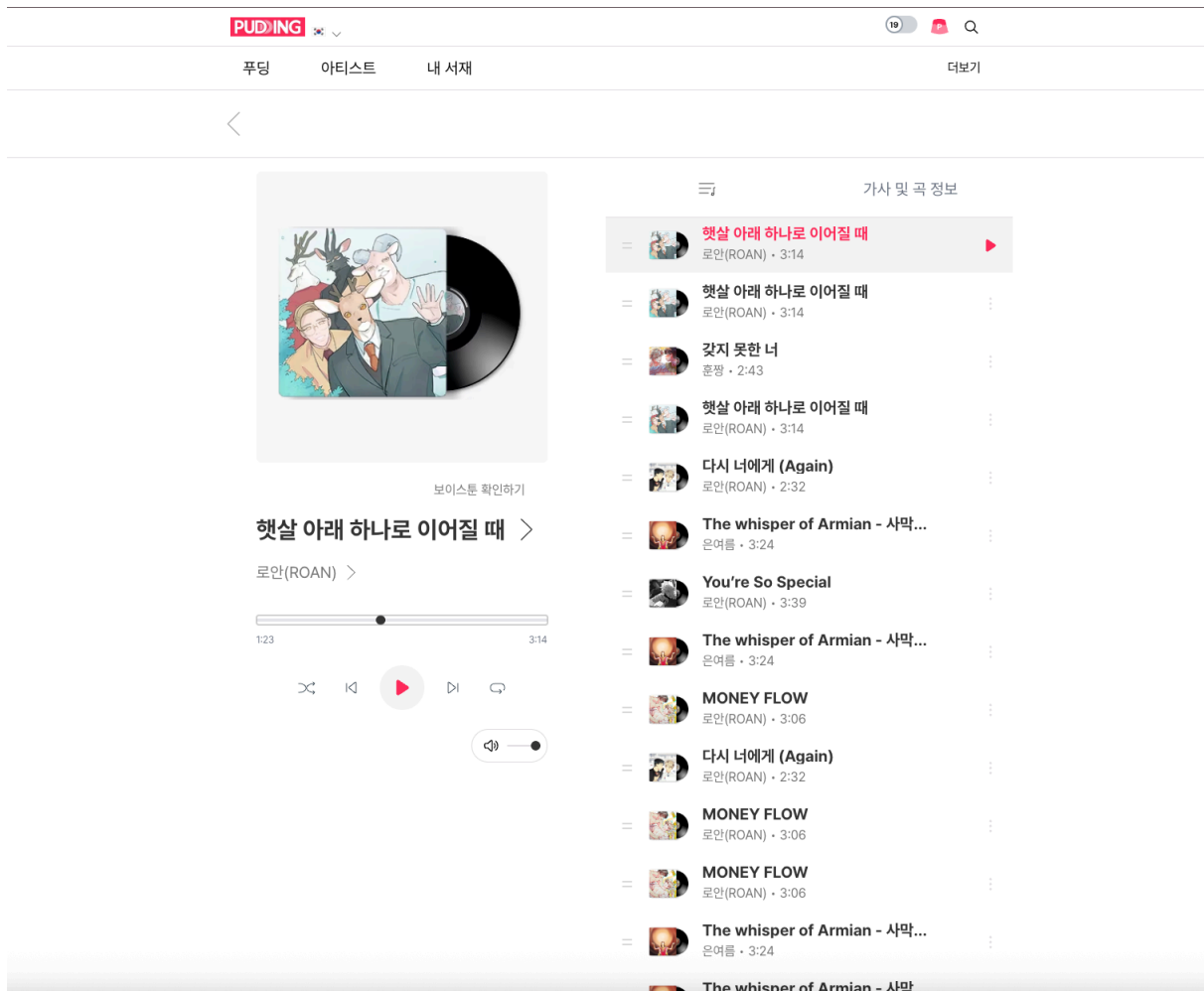
장르 ROCK

♡ 2

이미지 2. 뮤직 상세 페이지

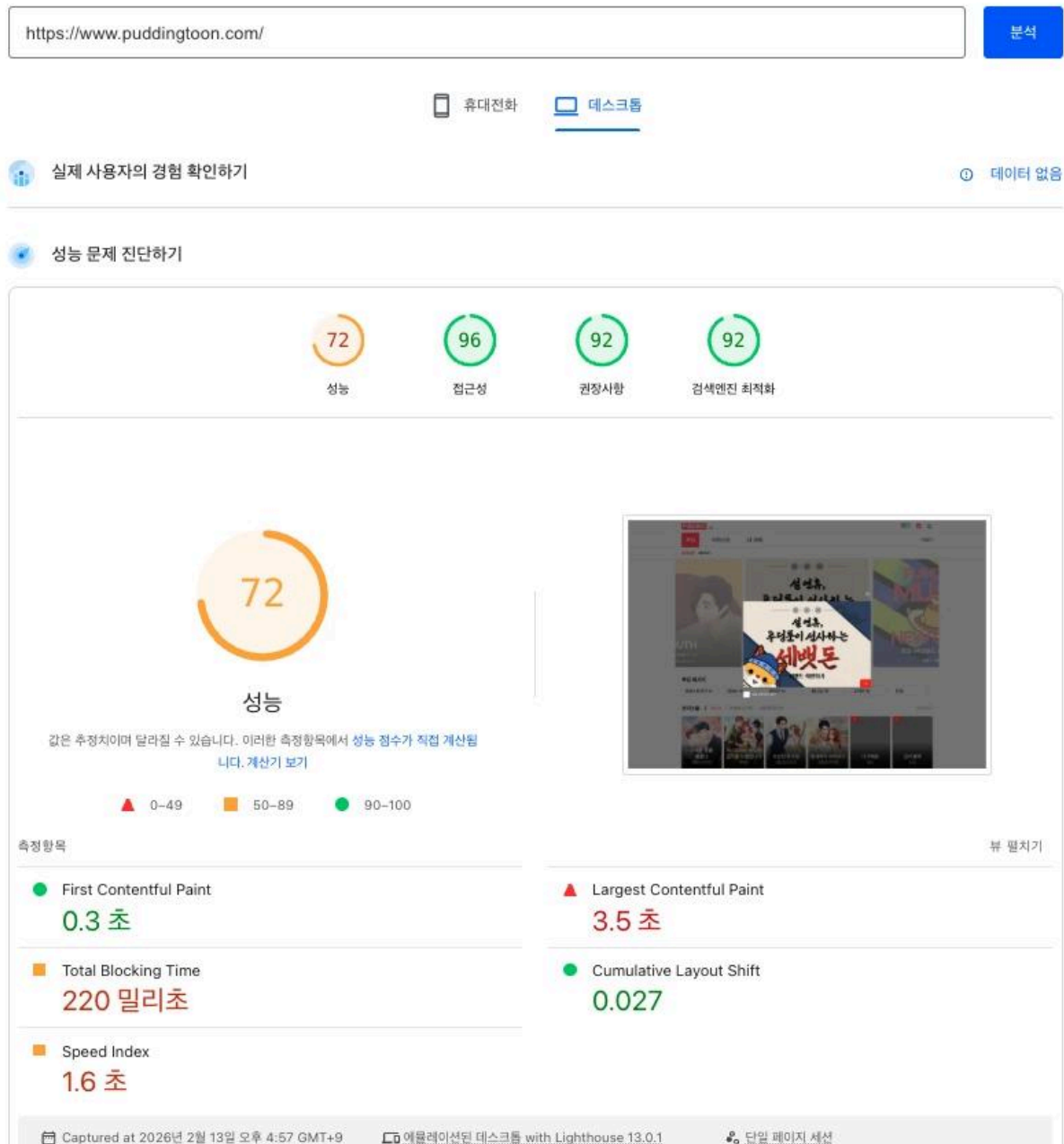
푸딩툰(React + Next.js) 안정화 & 뮤직(OST) 확장

6



이미지 3. 플레이어 상세 페이지

2026. 2. 13. PM 4:57:18의 보고서



이미지 4. 라이트하우스 사이트 성능 개선

